

```
try:
    x=8
    print(x)
except:
    print("an exception")

a=int(input("enter the value of a:"))
b=int(input("enter the value of b:"))
print(a/b)
8
```

```
try:
    print(x)
except:
    print("an exception")
an exception
```

```
a=int(input("enter the value of a:"))
b=int(input("enter the value of b:"))
print(a/b)
enter the value of a:2
enter the value of b:1
2.0
```

enter the value of a:3

enter the value of b:0

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 28, in <module>

print(a/b)

~^~

ZeroDivisionError: division by zero

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter the value of a:2

enter the value of b:t

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 27, in <module>

b=int(input("enter the value of b:"))

ValueError: invalid literal for int() with base 10: 't'

```
try:
    a=int(input("enter the value of a:"))
    b=int(input("enter the value of b:"))
    print(a/b)
except ValueError:
    print("please enter valid interger values")
except ZeroDivisionError:
    print("cannot divide by zero")
```

enter the value of a:2

enter the value of b:3

0.6666666666666666

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter the value of a:3

enter the value of b:0

cannot divide by zero

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter the value of a:3

enter the value of b:t

please enter valid interger values

```
try:
    number=int(input("enter a number:"))
    result=100/number
    print(result)
except(ValueError,ZeroDivisionError) as error:
    print("error:",error)
```

enter a number:3

33.333333333333336

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter a number:0

error: division by zero

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter a number:t

error: invalid literal for int() with base 10: 't'

enter a number:4.5

error: invalid literal for int() with base 10: '4.5'

```
try:
    print("Hello")
except:
    print("something went wrong")
else:
    print("nothing went wrong")
```

Hello

nothing went wrong

```
try:
    number=int(input("enter a number:"))
    result=100/number
except(ValueError,ZeroDivisionError) as error:
    print("error:",error)
else:
    print("Result:",result)
```

enter a number:2

Result: 50.0

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter a number:0

error: division by zero

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter a number:t

error: invalid literal for int() with base 10: 't'

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter a number:8.9

error: invalid literal for int() with base 10: '8.9'

```
try:
    f=open("students.txt")
```

```

try:
    f.write("Hello world")
except:
    print("something went wrong when writing to the file")
finally:
    f.close()
except:
    print("something went wrong when reading to the file")
something went wrong when reading to the file

```

```

try:
    f=open("students.txt","r")
    try:
        f.write("Hello world")
    except:
        print("something went wrong when writing to the file")
    finally:
        f.close()
except:
    print("something went wrong when reading to the file")

```

something went wrong when reading to the file

```

try:
    f=open("students.txt","w")
    try:
        f.write("Hello world")
    except:
        print("something went wrong when writing to the file")
    finally:
        f.close()
except:
    print("something went wrong when reading to the file")

```

no op

```

with open("stduent.txt","w") as file:
    data=file.read()
try:
    f=open("students.txt","w")
    try:
        f.write("Hello world")
    except:

```

```

        print("something went wrong when writing to the file")
    finally:
        f.close()
except:
    print("something went wrong when reading to the file")

```

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 56, in <module>

data=file.read()

io.UnsupportedOperation: not readable

```

#raise-Explicitly raising an exception
age=int(input("enter the age:"))
if age<0:
    raise ValueError("Age cannot be negative")
print("Age",age)

```

enter the age:20

Age 20

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter the age:-20

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 70, in <module>

raise ValueError("Age cannot be negative")

ValueError: Age cannot be negative

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter the age:t

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 68, in <module>

age=int(input("enter the age:"))

ValueError: invalid literal for int() with base 10: 't'

```

#raise an exception if marks are not between 0 and 100
marks=int(input("enter marks"))
if marks<0 or marks>100:
    raise ValueError("Marks must be between 0 and 100")

```

enter marks-2

Traceback (most recent call last):

```
File "c:\APPClass\lab5.py", line 76, in <module>
    raise ValueError("Marks must be between 0 and 100")
```

ValueError: Marks must be between 0 and 100

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter marks99

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter marks109

Traceback (most recent call last):

```
File "c:\APPClass\lab5.py", line 76, in <module>
    raise ValueError("Marks must be between 0 and 100")
```

ValueError: Marks must be between 0 and 100

```
#assert statement is used to test if a condition is true
#if the condition is false.an asserterror exception is raised
age=20
assert age>=21
print("Eligible")
```

Traceback (most recent call last):

```
File "c:\APPClass\lab5.py", line 81, in <module>
    assert age>=21
           ^^^^^^^
```

AssertionError

```
age=20
assert age>=18
print("Eligible")
Eligible
```

```
def calculate_average(marks):
    assert len(marks)>0,"Marks list cannot be empty"
    return sum(marks)/len(marks)
marks=[2,3,5,6]
marks1=[]
print(calculate_average(marks))
```

```
print(calculate_average(marks1))
```

4.0

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 89, in <module>

```
print(calculate_average(marks1))
```

~~~~~

File "c:\APPClass\lab5.py", line 84, in calculate\_average

```
assert len(marks)>0,"Marks list cannot be empty"
```

^^^^^^^^^^^^^^^^

AssertionError: Marks list cannot be empty

```
#user defined exception
class InvalidMarksError(Exception):
    pass
marks=int(input("enter marks:"))
try:
    if marks<0 or marks>100:
        raise InvalidMarksError("Marks must be 0 and 100")
    print("Marks:",marks)
except InvalidMarksError as error:
    print("Error:",error)
```

enter marks:90

Marks: 90

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter marks:-9

Error: Marks must be 0 and 100

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter marks:109

Error: Marks must be 0 and 100

PS C:\APPClass> & "C:\Program Files\Python314\python.exe" c:/APPClass/lab5.py

enter marks:t

Traceback (most recent call last):

File "c:\APPClass\lab5.py", line 94, in <module>

```
marks=int(input("enter marks:"))
```

ValueError: invalid literal for int() with base 10: 't'